sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Organizational Culture and Team Well-Being



Gemma Catolino | 2026-2027

Sustainable Software Engineering: From Human Factors to Collective Systems

Individual developers can be motivated, ethical, and competent, but they still operate *inside* a larger system: the organization.

If that system:

- rewards only speed,
- ignores emotional impact,
- tolerates toxic communication,

then individual good intentions are gradually eroded.

In previous lecture, we discussed how **well-being, fairness, and human debt** affect people and their work.

In this lecture, we ask:

What kind of culture is needed so that these human values are not exceptions, but the normal way of working?

This is why organizational culture, team climate, and DevOps practices are central to **sustainable software engineering**. Schein (2010); Forsgren et al. (2018)

PART 1 – ORGANIZATIONAL CULTURE

Sustainable Software Engineering: What We Mean by “Organizational Culture”

Organizational culture is **not** a list of slogans on a wall.

It is the **shared pattern of assumptions, values, and habits** that people use to decide:

- what is considered “good” work,
- who is respected,
- how conflicts are handled,
- what you can safely say out loud.

Edgar Schein (2010) defines culture as:

“A pattern of shared basic assumptions learned by a group as it solved its problems of external adaptation and internal integration.”

Culture is *learned*, *shared*, and *taught* to newcomers. It emerges over time from real decisions, not from marketing.



Sustainable Software Engineering: Schein's Three Levels of Culture in Software Teams

Schein describes culture as three layers:

1. **Artifacts** — visible elements:

- stand-up meetings, retrospectives, CI dashboards, review processes;
- Slack/Teams etiquette, how incidents are discussed, who speaks in meetings.

2. **Espoused Values** — what the organization *claims* it believes:

- "We are agile", "We care about quality", "People are our greatest asset".

3. **Underlying Assumptions** — what actually drives behaviour:

- "We never push back on unrealistic deadlines",
- "Senior devs are always right",
- "It's safer to stay silent than to question decisions".

Sustainability exists when **artifacts**, **values**, and **assumptions** are *aligned*.

If the organization says "we care about well-being" but rewards only overtime, culture is unsustainable. (Schein (2010))

Sustainable Software Engineering: Why Culture Is a Sustainability Mechanism

Culture influences all aspects of sustainability:

- **Technical** – determines whether teams invest in refactoring, documentation, and testing, or repeatedly cut corners.
- **Social** – shapes how information flows, how conflicts are handled, and whether knowledge is shared or hoarded.
- **Human** – affects belonging, stress, and the sense of meaning.
- **Organizational** – impacts retention, onboarding costs, and capacity to innovate.

If culture encourages short-term heroics and permanent urgency, it consumes people faster than it can replace them.

If culture supports learning, balance, and mutual support, it makes human energy **semi-renewable**: people can recover, grow, and stay.

(Edmondson (2018); Lenberg et al. (2015))

Sustainable Software Engineering: Empirical Evidence: Culture and Performance

Several empirical studies in software engineering demonstrate that organizational culture and team climate directly affect performance, productivity, and sustainability.

- **Lenberg et al. (2015)** found that *human and organizational factors*—such as trust, communication, and leadership—are cited more often than purely technical ones as major industrial challenges.
- **Forsgren, Humble & Kim (2018)**, through the *Accelerate* research program, showed that *generative cultures* (characterized by trust, transparency, and learning from failure) strongly correlate with higher deployment frequency, faster recovery, and better software quality.
- **Fagerholm & Münch (2012)** introduced the concept of **Developer Experience (DX)**, defining it as the developers' holistic perception of their work environment, tools, and culture. Their work established that positive DX—rooted in autonomy, clarity, and supportive culture—enhances satisfaction, creativity, and sustained engagement.
- **Tulili et al. (2022)** conducted a systematic mapping study on *burnout in software engineering*, highlighting that unsustainable workloads, poor communication, and lack of recognition are major predictors of exhaustion and attrition.

Sustainable Software Engineering: Cultural Archetypes in Software Organizations

We can loosely classify organizations into cultural archetypes that often appear in SE:

1. **Hero Culture**

- A few "rock stars" or "firefighters" save every release.
- Knowledge is concentrated; processes are improvisational.
- Short-term wins, long-term fragility and burnout.

2. **Bureaucratic Culture**

- Many rules; decision-making is centralized.
- Stability is high, but adaptation is slow; people feel constrained.

3. **Learning Culture**

- Mistakes are analyzed without punishment.
- Feedback is welcomed; reflection is normalized.
- Better suited for complex, changing environments.

4. **DevOps / Generative Culture**

- Cross-functional collaboration and shared ownership.
- Data-driven decisions, rapid feedback loops, and continuous improvement.

Sustainable software engineering tends to correlate with **learning** and **DevOps** cultures.

(Kim et al. (2016); Forsgren et al. (2018))

Sustainable Software Engineering: Team Climate: Culture in Daily Life

If culture is the deep structure, **team climate** is how it feels *today*.

Team climate includes:

- how people talk to each other,
- whether questions are welcomed or mocked,
- how workload is shared,
- whether people feel respected.

Two teams in the same company can have very different climates.

A positive climate supports curiosity and collaboration; a negative climate generates withdrawal and quiet quitting.

Climate is the **first place** where unsustainability becomes visible:

people get tired, irritated, or stop caring.

For sustainable engineering, monitoring climate is as important as monitoring CPU usage.

(Forsgren et al. (2018); Edmondson (2018))

PART 2 – TEAM WELL-BEING AND PRACTICES

Sustainable Software Engineering: Team Well-Being as a Collective Resource

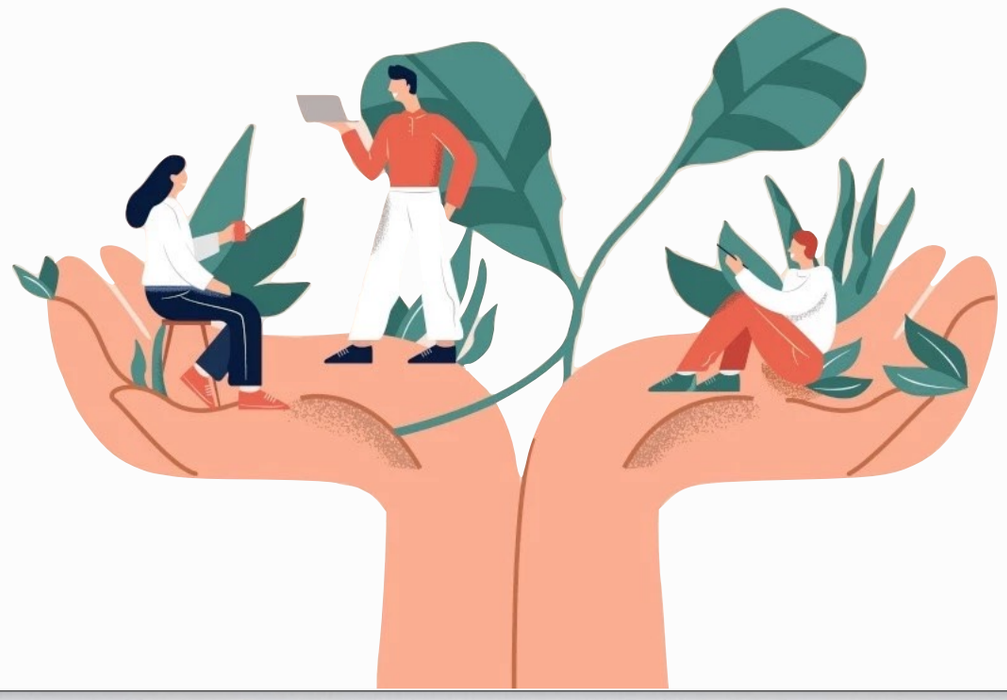
In previous lecture, we looked at individual well-being.

At the team level, we can think of **well-being as a shared resource**:

- It affects the *capacity* of a team to handle incidents, complex tasks, and change.
- It influences *how long* a team can sustain a given level of effort before quality drops.
- It determines whether people *stay* and grow, or leave and take their knowledge away.

Studies (e.g., Graziotin et al. 2018; Rahal et al. 2023) show that happier, less burned-out developers are more effective problem-solvers and produce better code.

Team well-being is, therefore, a **predictor of technical quality**, not a "nice to have".

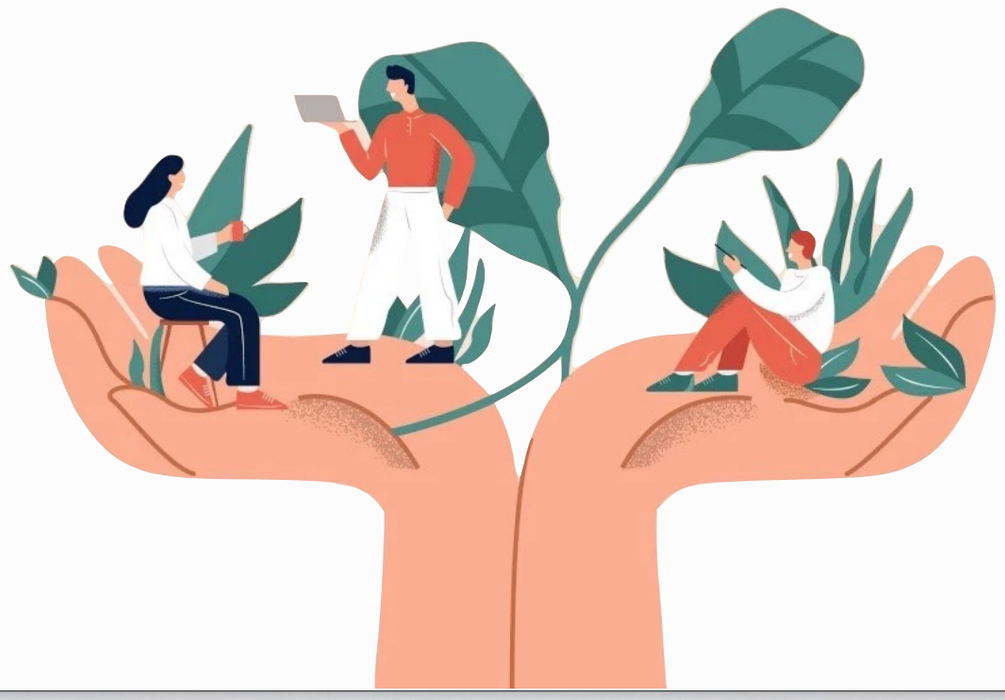


Sustainable Software Engineering: Sources of Team Well-Being (Organizational Perspective)

From an organizational point of view, key determinants of team well-being are:

- **Workload management:** realistic deadlines, sustainable pace, and the ability to say "no".
- **Autonomy:** teams can decide *how* to implement solutions, choose tools, and improve their process.
- **Growth opportunities:** time and support for learning new skills and technologies.
- **Social support:** peers and managers are accessible and responsive.
- **Recognition:** invisible work (mentoring, debugging, maintenance) is valued.

Without these conditions, even technically perfect processes become humanly unsustainable.

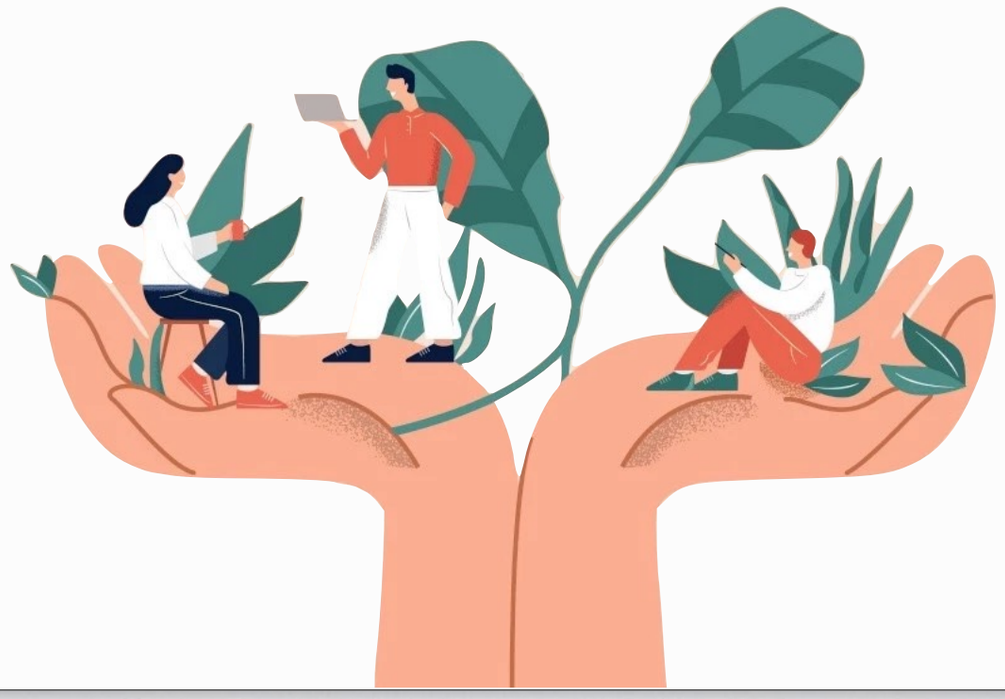


Sustainable Software Engineering: Organizational Practices that Support Well-Being

Sustainable organizations embed well-being into their **operational practices**, for example:

- **Transparent planning:** teams see the full picture and understand trade-offs.
- **Retrospectives with real follow-up:** identified issues (e.g., too many interrupts, unclear responsibilities) lead to actual changes.
- **Rotation:** high-stress roles (like on-call) are rotated and compensated fairly.
- **Clear escalation paths:** people know when and how to ask for help.
- **Regular 1-to-1s:** managers check not only progress but also stress levels, support needs, and future plans.

These practices turn "we care about people" from a statement into a **repeatable process**.



Sustainable Software Engineering: Measuring Well-Being Without Becoming Creepy

Monitoring well-being is tricky: if done badly, it becomes surveillance; if done thoughtfully, it becomes reflection.

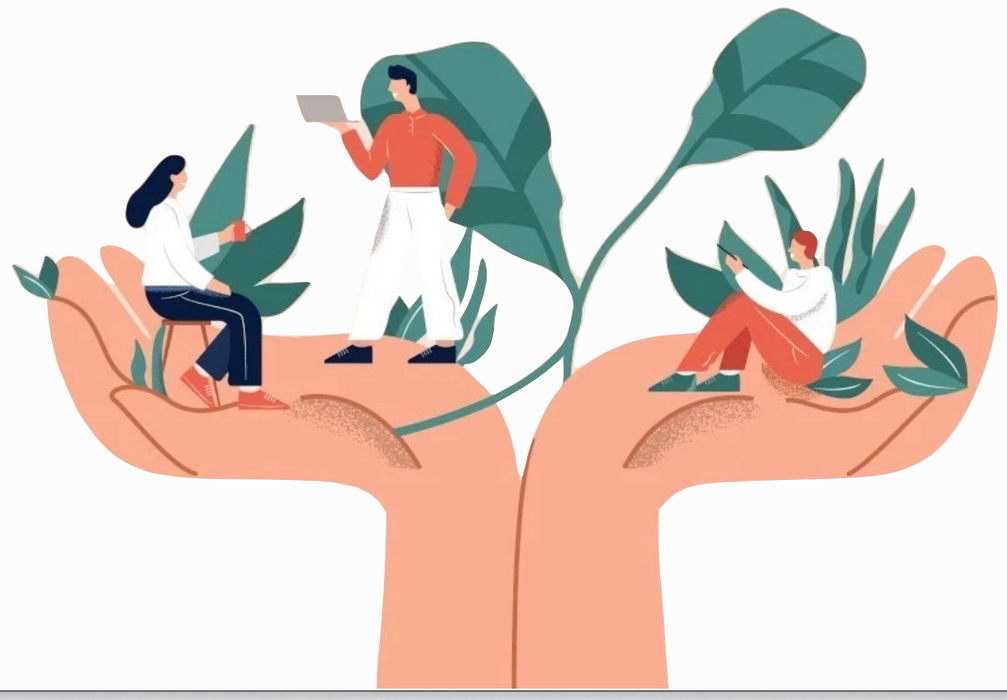
Useful **aggregated** indicators include:

- short, anonymous "pulse" surveys (e.g., stress, workload fairness, clarity of goals),
- review and issue latency (when delays become chronic, there may be overload),
- turnover and exit interview themes,
- number of after-hours commits or meetings (trend, not individual blame).

Numbers must always be interpreted **with** teams, not used **against** them.

The goal is to create shared awareness and corrective action, not to rank people.

(Forsgren et al. (2018); Rahal et al. (2023))



PART 3 – DEVOPS: WHAT IT IS AND WHY IT MATTERS FOR SUSTAINABILITY

Sustainable Software Engineering: What DevOps Is (and Is Not)

DevOps is often misunderstood as “just automation” or “just CI/CD”.

In reality, DevOps is: A **set of cultural principles and technical practices** that bring development and operations together to deliver software rapidly, reliably, and safely.

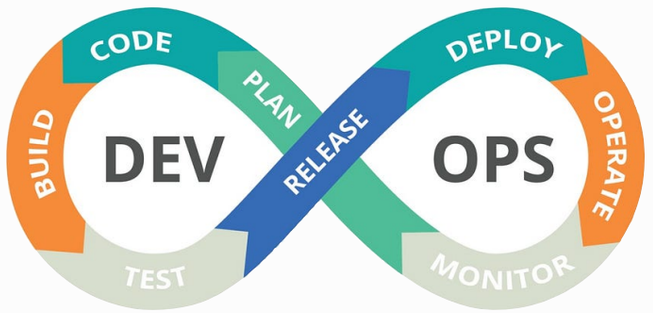
It explicitly aims to break the wall between “those who build it” and “those who run it”.

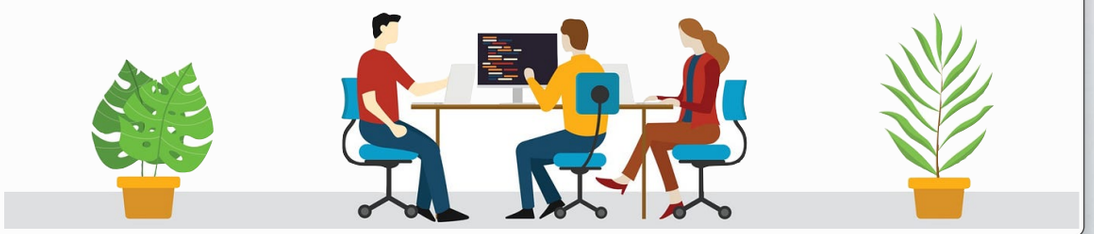
Core DevOps beliefs:

- collaboration beats silos,
- small, frequent changes beat big risky releases,
- learning from failure beats hiding it,
- automation is used to remove pain, not to replace responsibility.

DevOps is therefore a **socio-technical redesign** of how we build and run software.

(Kim et al. (2016); Beyer et al. (2016))





Sustainable Software Engineering: DevOps Principles and Cultural Shifts

Kim et al. (2016) summarise DevOps culture through the "Three Ways":

1. **Flow:**

- optimise the end-to-end flow of work from idea to user;
- reduce hand-offs and waiting;
- design pipelines that are fast and predictable.

2. **Feedback:**

- shorten feedback loops from systems and users;
- make failures visible;
- empower teams to react quickly.

3. **Continuous Learning & Experimentation:**

- create a culture of curiosity;
- encourage safe experiments;
- institutionalise learning from incidents.

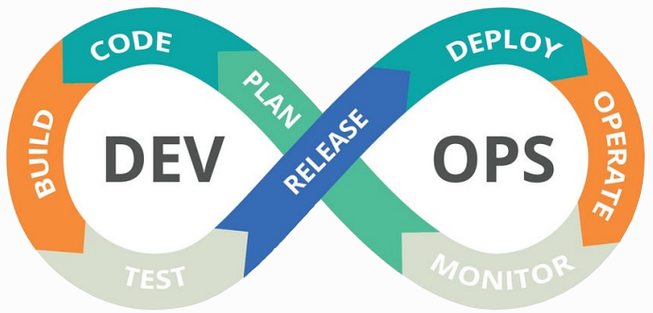
These principles require deep cultural change: trust, transparency, and shared ownership.

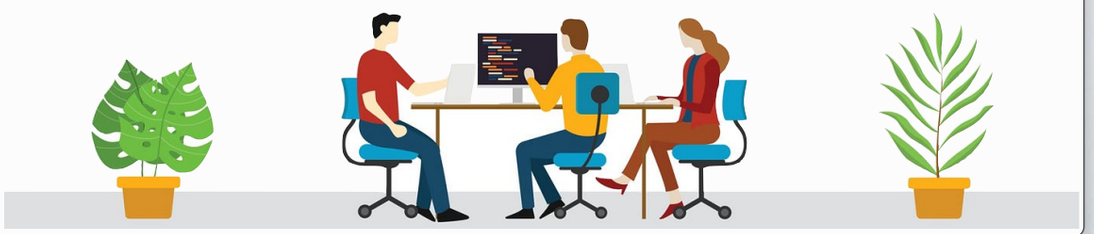
Sustainable Software Engineering: DevOps and Human Sustainability

From a human perspective, DevOps can support sustainability if implemented with the right values:

- **Shared ownership** reduces blame: incidents become collective challenges, not witch hunts.
- **Automation** removes repetitive, high-stress tasks (e.g., manual deployments, fragile scripts), freeing cognitive capacity.
- **Blameless post-mortems** protect psychological safety, so people can talk openly about problems.
- **Smaller batches** mean fewer all-nighters and emergency releases.

Forsgren et al. (2018) show that high-DevOps-maturity teams have *lower burnout* and *higher job satisfaction* than low-maturity teams. DevOps can be a **human sustainability pattern**, if culture stays aligned.





Sustainable Software Engineering: DevOps and Technical Sustainability

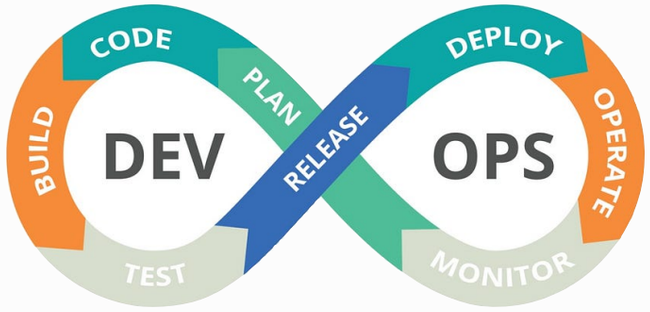
DevOps also contributes to technical sustainability:

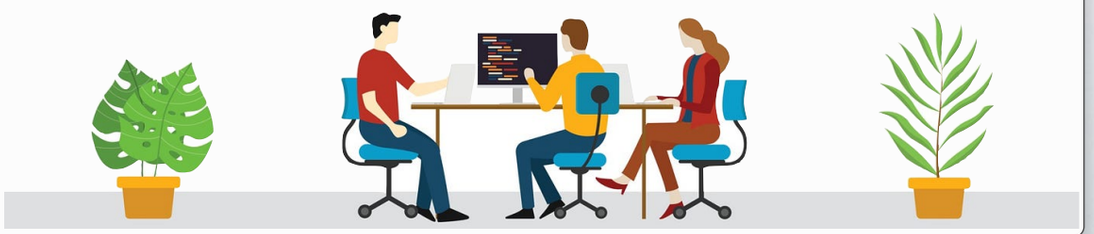
- continuous integration and delivery reduce integration hell,
- smaller changes are easier to test and roll back,
- observability tools (logs, metrics, tracing) help understand complex failures,
- infrastructure as code documents environments in a reproducible way.

This improves maintainability and reduces the temptation to "hack something quickly in production".

Technical and human sustainability reinforce each other: less chaos → less stress → more thoughtful engineering.

(Beyer et al. (2016); Forsgren et al. (2018))



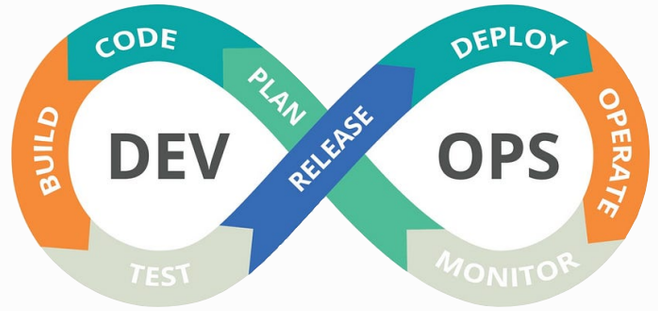


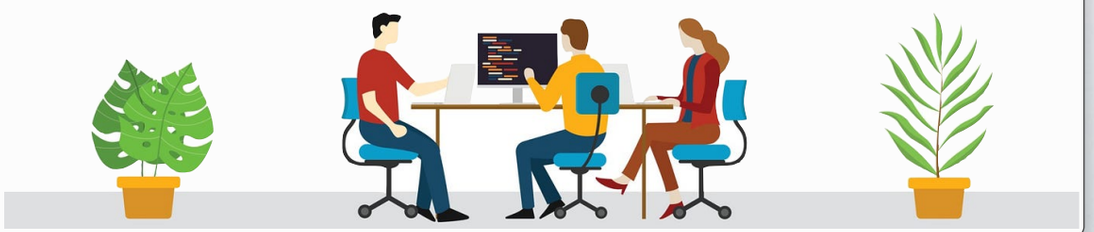
Sustainable Software Engineering: DevOps and Environmental Sustainability

DevOps also interacts with **environmental and economic sustainability**:

- smaller incremental releases avoid huge, wasteful build/test cycles,
- auto-scaling infrastructure reduces energy use and cost when demand is low,
- performance monitoring can identify inefficient components that waste CPU, RAM, or cloud budget,
- efficient pipelines reduce the need for large, duplicated test environments.

If organizations connect these technical practices to explicit environmental goals, DevOps becomes part of a broader sustainability strategy: fewer wasted resources, fewer wasted hours, fewer burned-out people.





PART 4 – GLOBAL CULTURE, DISPERSION, AND COMMUNITY RISKS

Sustainable Software Engineering: Cultural Dispersion in Global Software Teams

Modern software projects are rarely co-located.

People work across:

- different countries and languages,
- different companies or clients,
- different time zones and social norms.

Lambiase et al. describe this as **cultural dispersion**.

Dispersion increases diversity of perspectives (good for innovation) but also:

- raises the risk of misinterpretation and misunderstanding,
- complicates coordination and shared ownership,
- can make some contributors less visible than others.

If not handled deliberately, dispersion can damage both culture and well-being.



Sustainable Software Engineering: How Dispersion Impacts Communities (Empirical Insights)

Teams are no longer co-located: they work across continents, languages, and time zones. This condition, called **cultural dispersion** (*Lambiase et al., 2024*), profoundly influences how people collaborate, learn, and sustain their energy over time.

Cultural dispersion introduces both opportunity and risk. Diversity of experience and perspective often stimulates creativity and richer design discussions. Distributed work can also make projects more resilient and inclusive, drawing on different cultural strengths.

Yet distance, language, and differing assumptions can just as easily fragment communication and trust. Time-zone gaps reduce informal learning; remote contributors can feel invisible; coordination becomes slower and more stressful.

Empirical studies confirm this tension.

In *There and Back Again?* (*Lambiase et al., 2022*), moderate dispersion increased innovation, but extreme dispersion reduced coordination quality.

In *Good Fences Make Good Neighbours?* (*Lambiase et al., 2022*), dispersion was linked to "community smells" such as silos and radio silence—early symptoms of social unsustainability.

Sustainable Software Engineering: A Framework for Dealing with Cultural Dispersion

In 2024, *Lambiase et al.* proposed a framework that treats cultural dispersion as a **design property** of socio-technical systems rather than a logistical obstacle.
Drawing on cross-cultural psychology (*Hofstede, 2010*) and organizational theory, the framework distinguishes three layers that must be engineered together.

At the **cognitive layer**, teams must surface and reconcile different mental models of what "good work" means. Shared onboarding, cross-site workshops, and joint retrospectives help create common understanding.
At the **interactional layer**, communication norms need to be explicit: how to disagree, how quickly to reply, which language to use for documentation.
Finally, at the **structural layer**, fairness and visibility must be designed into processes—rotating meeting times, sharing leadership roles, and making contributions equally visible across sites.

These practices transform dispersion into a sustainable pattern. They protect **human sustainability** by reducing isolation and fatigue, strengthen **social sustainability** by ensuring inclusion and trust, and enhance **technical sustainability** by aligning team boundaries with architectural boundaries.

PART 5 – MONITORING, DIAGNOSING, AND CONTINUOUS IMPROVEMENT

Sustainable Software Engineering: Why Monitoring Culture Is Necessary (and Difficult)

Culture and climate are dynamic: they change with growth, crises, and leadership shifts. Without deliberate monitoring, organizations realise there is a cultural problem only when people are already leaving or quality has collapsed.

However, monitoring culture is delicate:

- too little → blind to problems;
- too much or badly done → feels like surveillance and erodes trust.

The goal is *not* to spy on individuals but to detect **patterns** that affect sustainability and to start informed discussions.

(Forsgren et al. (2018); Edmondson (2018))



Sustainable Software Engineering: Best Practices for Monitoring Culture and Well-Being

Some practical practices for ethically monitoring culture:

- **Short, anonymous "pulse" surveys** on stress, clarity, and inclusion.
- **Team-level workload data** (e.g., average review load, on-call frequency, after-hours work as a trend).
- **Collaboration graphs** based on code reviews or communication (not to judge individuals but to detect silos).
- **Qualitative data** from retrospectives and one-to-one meetings, used in aggregated form.

Important constraints:

- explain clearly *why* data is collected and *how* it will be used;
- never use these metrics to punish individuals;
- always validate findings in conversation with teams.



Sustainable Software Engineering: Continuous Cultural Improvement Loop

Culture, like software, requires continuous integration and delivery.

Without active maintenance, cultural drift occurs — people revert to silos, distrust, or burnout-inducing habits.

A sustainable organization treats culture as a **living system**.

This means running an ongoing cycle:

1. **Observe** — collect evidence from surveys, retrospectives, and communication networks.
2. **Reflect** — create safe spaces where teams interpret this evidence collectively.
3. **Experiment** — test small, behavioural improvements (e.g., changing meeting formats, mentoring rotations).
4. **Reinforce** — embed what works into daily processes and rituals.

But sustaining this loop requires *leadership energy*.

Without leaders who listen, protect time, and allocate resources, the loop breaks at step 2 or 3.

Sustainable Software Engineering: Leadership as the Amplifier of Culture

Leadership is the **mechanism that keeps cultural improvement alive**.

While culture emerges from everyone's behaviour, leaders define its *direction* and *speed* of evolution. In sustainable software organizations, leaders:

- **Model behaviour.**

  People copy what leaders do, not what they say.

  When leaders openly admit mistakes or ask for feedback, they legitimise learning and humility across the team.

- **Distribute authority.**

  They create psychological safety by letting teams own their processes.

  This supports DevOps principles of autonomy and shared responsibility.

- **Align metrics with values.**

  Unsustainable leadership focuses on delivery speed; sustainable leadership balances *throughput* with *well-being, fairness, and learning*.

Sustainable Software Engineering: Leadership as the Amplifier of Culture

- **Protect slack time.**

  They treat rest, retrospectives, and refactoring as necessary investments — not as "waste".

- **Enable cross-cultural empathy.**

  In dispersed or diverse teams, leaders act as *cultural translators* — making sure communication, recognition, and opportunities are equitable across geographies.

Thus, leadership is not only hierarchical; it's **cultural infrastructure**.

It ensures that human and technical sustainability remain coupled over time.

(Edmondson (2018); Forsgren et al. (2018); Lambiase et al. (2024))

Sustainable Software Engineering: Implications for Sustainable Software

Taken together, these ideas support a broader view of sustainable SE:

- Code and infrastructure are only one layer of sustainability.
- Human beings, and the cultures they form, are the long-lived core of any project.
- DevOps shows that we can design socio-technical systems that are fast *and* humane.
- Monitoring and reflection help maintain alignment as projects grow and globalize.
- Community smells and social debt (next lecture) give us analytical tools to see where things are breaking.

Sustainable Software Engineering: References

1. Schein, Edgar H. *Organizational culture and leadership*. Vol. 2. John Wiley & Sons, 2010.
2. Forsgren, Nicole, Jez Humble, and Gene Kim. *Accelerate: The science of lean software and devops: Building and scaling high performing technology organizations*. IT Revolution, 2018.
3. Edmondson, Amy C. *The fearless organization: Creating psychological safety in the workplace for learning, innovation, and growth*. John Wiley & Sons, 2018.
4. Kim, Gene, et al. *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations*. It Revolution, 2021.
5. Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: how Google runs production systems*. " O'Reilly Media, Inc.".
6. Lenberg, Per, Robert Feldt, and Lars Göran Wallgren. "Human factors related challenges in software engineering--an industrial perspective." *2015 ieee/acm 8th international workshop on cooperative and human aspects of software engineering*. IEEE, 2015.
7. Tulili, Tien Rahayu, Andrea Capiluppi, and Ayushi Rastogi. "Burnout in software engineering: A systematic mapping study." *Information and Software Technology* 155 (2023): 107116
8. Fagerholm, Fabian and Jürgen Münch. "Developer experience: Concept and definition." *2012 international conference on software and system process (ICSSP)*. IEEE, 2012
9. Lambiase, S., Catolino, G., Tamburri, D. A., Serebrenik, A., Palomba, F., & Ferrucci, F. (2022, May). Good fences make good neighbours? on the impact of cultural and geographical dispersion on community smells. In *Proceedings of the 2022 ACM/IEEE 44th International Conference on Software Engineering: Software Engineering in Society* (pp. 67-78).
10. Lambiase, S., Catolino, G., Pecorelli, F., Tamburri, D. A., Palomba, F., Van Den Heuvel, W. J., & Ferrucci, F. (2022, August). “There and Back Again?” on the influence of software community dispersion over productivity. In *2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)* (pp. 177-184). IEEE.
11. Lambiase, S., Catolino, G., Piana, B. D., Ferrucci, F., & Palomba, F. (2024, April). Dealing with cultural dispersion: A novel theoretical framework for software engineering research and practice. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society* (pp. 74-84).

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Organizational Culture and Team Well-Being



Gemma Catolino | 2026-2027